

Understanding Common Java Compiler Errors in the zyBook

Recognizing and Fixing Compiler Errors

Compiler errors occur when there is an error in your code syntax – it is not written the way Java expects it to be written.

So when you press the RUN button, instead of running the program, compiler errors are listed on the output screen.

TABLE OF CONTENTS

1) COMPILER ERROR: Semi-colon expected.....	2
2) COMPILER ERROR: Cannot find symbol.....	2
COMPILER ERROR: Cannot find symbol.....	3
COMPILER ERROR: Cannot find symbol.....	4
3) COMPILER ERROR: Illegal start of expression or Illegal start of type.....	5
COMPILER ERROR: Illegal start of expression or Illegal start of type.....	6
4) COMPILER ERROR: class, interface, or enum expected.....	7
5) COMPILER ERROR: reached end of file while parsing.....	8
6) COMPILER ERROR: '(' expected or ')' expected.....	9
COMPILER ERROR: '(' expected or ')' expected.....	10
7) COMPILER ERROR: unclosed string literal.....	11
8) COMPILER ERROR: variable might not have been initialized.....	12
9) COMPILER ERROR: incompatible types: typeA cannot be converted to typeB.....	13
10) DURING zyLab Testing, when you Submit for grading COMPILER ERROR: Exception in thread "main" java.util.NoSuchElementException.....	14

1) COMPILER ERROR: Semi-colon expected

Reason for error: Java statements must end in a semi-colon, and you forgot a semi-colon.

Example:

```
3 public class ErrorExample {  
4  
5     public static void main(String[] args) {  
6         int number;  
7         number = 5;  
8         System.out.println("number is " + number)  
9     }  
10 }
```

```
DESKTOP  CONSOLE  (+)  
ErrorExample.java:8: error: ';' expected  
    System.out.println("number is " + number)  
                                ^  
1 error  
→
```

The missing semi-colon will generally be needed on the line noted in the error message (circled above), but could also be on code within a couple of lines above that line.

In this case, you could fix this error by adding the missing semi-colon at the end of line 8.

```
System.out.println("number is " + number);
```

2) COMPILER ERROR: Cannot find symbol

Reason for error: The program cannot find the item (name/identifier) specified in the code.

- Possible cause:* You misspelled a variable name, class name, or keyword.

Example (misspelled the variable name **number**):

```
3 public class ErrorExample {  
4  
5     public static void main(String[] args) {  
6         int number;  
7         numbr = 5;  
8         System.out.println("number is " + number);  
9     }  
10 }
```

```
DESKTOP  CONSOLE  (+)  
ErrorExample.java:7: error: cannot find symbol  
    numbr = 5;  
    ^  
symbol:   variable numbr  
location: class ErrorExample  
1 error  
→
```

Fix this code by changing variable name on line 7 from **numbr** to **number**

```
number = 5;
```

COMPILER ERROR: Cannot find symbol

b. *Another possible cause:*

You are trying to access a variable or class that does not exist (because it has not been declared), or cannot be accessed (because it was not declared within the **main()** method).

Example (did not declare variable **sum** before trying to assign a value to it):

```
3 public class ErrorExample {  
4  
5     public static void main(String[] args) {  
6         int num1 = 4;  
7         int num2 = 5;  
8         sum = num1 + num2;  
9         System.out.println("sum is " + sum);  
10    }  
11 }
```

DESKTOP CONSOLE ⊕

```
ErrorExample.java:8: error: cannot find symbol  
    sum = num1 + num2;  
    ^  
symbol:   variable sum  
location: class ErrorExample  
ErrorExample.java:9: error: cannot find symbol  
    System.out.println("sum is " + sum);  
                             ^  
symbol:   variable sum  
location: class ErrorExample  
2 errors  
→
```

This causes **two** errors, because **sum** is used on two **different** lines of code.

Fix this code by adding a declaration at the top of the **main()** method, along with the other variable declarations (i.e. before you use the variable in lines 8 and 9):

```
int sum;
```

COMPILER ERROR: Cannot find symbol

- c. Another possible cause: You are missing an **import** statement.

Example

(using the **Scanner** class in your code, without importing the **java.util.Scanner** class):

```
1 public class ErrorExample {
2
3     public static void main(String[] args) {
4         int num;
5         Scanner scnr = new Scanner(System.in);
6
7         System.out.println("Enter an integer number:");
8         num = scnr.nextInt();
9         System.out.println("num is " + num);
10    }
11 }
```

```
ErrorExample.java(5) error: cannot find symbol
    Scanner scnr = new Scanner(System.in);
    ^
symbol:   class Scanner
location: class ErrorExample
ErrorExample.java(5) error: cannot find symbol
    Scanner scnr = new Scanner(System.in);
    ^
symbol:   class Scanner
location: class ErrorExample
2 errors
→
```

This causes **two** errors, because **Scanner** is used twice within the **same** line of code, but the **Scanner** class code is not available for use.

Fix this code by adding this import statement at the top of the file, **above** the class, on the first line:

```
1 import java.util.Scanner;
2
3 public class ErrorExample {
4
5     public static void main(String[] args) {
6         int num;
7         Scanner scnr = new Scanner(System.in);
8
9         System.out.println("Enter an integer number:");
10        num = scnr.nextInt();
11        System.out.println("num is " + num);
12    }
13 }
```

3) COMPILER ERROR: Illegal start of expression or Illegal start of type

Reason for error: The program does not follow Java's required code structure syntax.

To fix, make sure your curly braces { } and parentheses () are all in the right place.

For now, all of your program code (besides **import** statements), must appear **between** the open and closing curly braces of the **main()** method.

Example

(The output statement is listed after the **main()** method's closing curly brace):

```
1 public class ErrorExample {  
2  
3     public static void main(String[] args) {  
4         System.out.println("Hello");  
5     }  
6  
7     System.out.println("Goodbye");  
8 }
```

```
ErrorExample.java:7: error: <identifier> expected  
    System.out.println("Goodbye");  
                   ^  
ErrorExample.java:7: error: illegal start of type  
    System.out.println("Goodbye");  
                   ^  
2 errors  
→
```

Note that the system is not sure what the error is.

So it gives you two possible different errors for the same code line.

In this case, the second **println()** statement on line 7 appears **after** of the **main()** method's closing curly brace.

Fix this code by moving the code on line 7 to line 5 (inside the **main()** method curly braces):

```
1 public class ErrorExample {  
2  
3     public static void main(String[] args) {  
4         System.out.println("Hello");  
5         System.out.println("Goodbye");  
6     }  
7  
8 }
```

COMPILER ERROR: Illegal start of expression or Illegal start of type

This error can also occur when you have characters inside the parentheses of a **println()** statement that should be part of a **String** enclosed in double quotes, but appear outside the double quotes instead.

Example:

```
3 public class ErrorExample {
4     public static void main(String[] args) {
5
6         System.out.println([1] "Hello");
7
8     }
9 }
```

DESKTOP CONSOLE +

```
ErrorExample.java:6: error: illegal start of expression
    System.out.println([1] "Hello");
                        ^
ErrorExample.java:6: error: ';' expected
    System.out.println([1] "Hello");
                        ^
```

2 errors

→

To fix this error, ensure all characters that should be displayed on the screen in line 6 are included **within** the double quotes:

```
System.out.println("[1] Hello");
```

4) COMPILER ERROR: class, interface, or enum expected

Reason for error:

Usually caused by a misplaced curly brace – most often when there is an **extra** closing brace in your code.

Make sure all of your code is **within** a pair of curly braces.

Note that when you click on an open curly brace, it will be highlighted, along with the closing brace that Java thinks it is paired with.

Example (Extra closing brace in program):

```
3 public class ErrorExample {  
4  
5     public static void main(String[] args) {  
6         System.out.println("Hello!");  
7         System.out.println("Goodbye");  
8     }  
9 }  
10  
11 }
```

DESKTOP CONSOLE +

```
ErrorExample.java:11: error: class, interface, or enum expected  
}  
^  
1 error  
→
```

The error is actually **above** the referenced line.

But the Java compiler did not figure it out until it encountered one too many curly braces.

Fix this code by removing extra closing curly brace on line 9.

5) COMPILER ERROR: reached end of file while parsing

Reason for error:

Usually caused by a misplaced curly brace – most often when there is a **missing** closing brace } in your code.

Again, make sure all of your code is within a pair of curly braces.

Example (Missing closing brace for **main** method):

```
3 public class ErrorExample {  
4  
5     public static void main(String[] args) {  
6         System.out.println("Hello!");  
7         System.out.println("Goodbye");  
8     }  
9 }
```

DESKTOP CONSOLE +

```
ErrorExample.java:9: error: reached end of file while parsing  
    }  
    ^  
1 error  
→
```

Again, the error is actually **above** the referenced line.

Every open curly brace must have a matching closing curly brace.

Initially your programs will have one set of open/closing curly braces for the class,
and one set of open/closing curly braces for the main() method.

This code is missing the closing curly brace for the **main()** method.

So fix this code by **adding** the missing closing brace for the **main()** method on line 8.

6) COMPILER ERROR: '(' expected or ')' expected

Reason for error:

You forgot a left or right parenthesis in your code.

For every left parenthesis in your program, you must have a matching right parenthesis.

Note that when you click on an open parenthesis, it will be highlighted, along with the closing parenthesis that Java thinks it is paired with.

Example (Missing right parenthesis for println() statement):

```
3 public class ErrorExample {
4
5     public static void main(String[] args) {
6         System.out.println("Hello!");
7         System.out.println("Goodbye";
8     }
9 }
```

DESKTOP CONSOLE +

```
ErrorExample.java:7: error: ')' expected
    System.out.println("Goodbye";
                        ^
1 error
→
```

Fix this code by adding the right parenthesis just before the semi-colon on line 7:

```
System.out.println("Goodbye");
```

Note that a missing open parenthesis can sometimes cause a more complicated looking error.

Example:

```
3 public class ErrorExample {
4
5     public static void main(String[] args) {
6         System.out.println("Hello!");
7         System.out.println"Goodbye");
8     }
9 }
```

DESKTOP CONSOLE +

```
ErrorExample.java:7: error: not a statement
    System.out.println"Goodbye");
                        ^
ErrorExample.java:7: error: ';' expected
    System.out.println"Goodbye");
                        ^
2 errors
→
```

The system cannot figure out the error, so it lists several possibilities.

In this case, fix this code by adding the left (opening) parenthesis just after the word **println** on line 7:

```
System.out.println("Goodbye");
```

COMPILER ERROR: '(' expected or ')' expected

Also note that sometimes this error occurs, but a missing parenthesis is not the issue.

Example:

```
3 public class ErrorExample {  
4     public static void main(String[] args) {  
5         int num = 5;  
6         System.out.println("num is " num);  
7     }  
8 }
```

DESKTOP CONSOLE (+)

```
ErrorExample.java:6: error: ')' expected  
    System.out.println("num is " num);  
                           ^  
ErrorExample.java:6: error: not a statement  
    System.out.println("num is " num);  
                           ^  
ErrorExample.java:6: error: ';' expected  
    System.out.println("num is " num);  
                           ^  
3 errors
```

Three different possible errors are listed, because the system is not sure what is wrong.

In this case, the error is a missing plus (+) between the String and the variable name, so the statement does not make sense in Java.

Fix by adding the missing + on line 6:

```
System.out.println("num is " + num);
```

7) COMPILER ERROR: unclosed string literal

Reason for error:

You started a string with a double quote, but you forgot the closing double quote at the end.

Example (Missing closing double quote):

```
3 public class ErrorExample {
4
5     public static void main(String[] args) {
6         System.out.println("Hello!);
7         System.out.println("Goodbye");
8     }
9 }
```

DESKTOP CONSOLE (+)

```
ErrorExample.java:6: error: unclosed string literal
    System.out.println("Hello!);
                        ^
1 error
→
```

Fix this code by adding the double quote after the **Hello!** on line 6:

```
System.out.println("Hello!");
```

NOTE: If you forget BOTH double quotes, the error can look different.

Example (Missing both double quotes):

```
3 public class ErrorExample {
4     public static void main(String[] args) {
5
6         System.out.println("Hello!");
7         System.out.println(Goodbye);
8     }
9 }
```

DESKTOP CONSOLE (+)

```
ErrorExample.java:7: error: cannot find symbol
    System.out.println(Goodbye);
                        ^
    symbol:   variable Goodbye
    location: class ErrorExample
1 error
```

Without any quotes, the system assumes that **Goodbye** is a variable name that was not declared.

Fix by adding the both double quotes on line 7:

```
System.out.println("Goodbye");
```

Note that the text turns orange when it is correctly enclosed in double quotes.

8) COMPILER ERROR: variable might not have been initialized

Reason for error:

You used a variable in a statement (displaying it, or within a calculation expression) before giving it an initial value.

Example:

```
1 public class ErrorExample {  
2  
3     public static void main(String[] args) {  
4         int num;  
5  
6         System.out.println("num is " + num);  
7     }  
8  
9 }
```

```
ErrorExample.java:6: error: variable num might not have been initialized  
    System.out.println("num is " + num);  
                                ^  
1 error  
→
```

To fix this error, either initialize the variable's value when you declare it on line 4:

```
int num = 5;
```

OR assign the variable's value sometime before you try to display it
(for example, from user input with a Scanner):

```
1 import java.util.Scanner;  
2  
3 public class ErrorExample {  
4  
5     public static void main(String[] args) {  
6         int num;  
7         Scanner scnr = new Scanner(System.in);  
8  
9         System.out.println("Enter an integer number:");  
10        num = scnr.nextInt();  
11        System.out.println("num is " + num);  
12    }  
13  
14 }
```

9) COMPILER ERROR: incompatible types: typeA cannot be converted to typeB

Reason for error:

You used the wrong variable **type** in an expression.

Example (Incorrectly assigning a **String** value to an **integer** variable):

```
1 public class ErrorExample {  
2  
3     public static void main(String[] args) {  
4         int num = "50";  
5         System.out.println("num is " + num);  
6     }  
7 }
```

```
ErrorExample.java:4: error: incompatible types: String cannot be converted to int  
    int num = "50";  
              ^  
1 error  
→
```

When digits are enclosed in double quotes, they become a String, instead of an integer.

Fix this code by removing the double quotes, to assign an integer value to variable **num** on line 4:

```
int num = 50;
```

10) DURING zyLab Testing, when you Submit for grading

COMPILER ERROR: Exception in thread "main" java.util.NoSuchElementException

Reason for error: The zyBook testing unit has provided the correct number of inputs, but your program reads *more input* than it should.

Example:

```
1  import java.util.Scanner;
2
3  public class ErrorExample {
4      public static void main(String[] args) {
5          Scanner scnr = new Scanner(System.in);
6          int num1;
7          int num2;
8          int sum;
9          System.out.println("Enter first number");
10         num1 = scnr.nextInt();
11         num1 = scnr.nextInt();
12         System.out.println("Enter second number");
13         num2 = scnr.nextInt();
14         sum = num1 + num2;
15         System.out.println("sum is " + sum);
16     }
17 }
```

1: Compare output ^

Compare output

```
Exception in thread "main" java.util.NoSuchElementException
    at java.base/java.util.Scanner.throwFor(Scanner.java:937)
    at java.base/java.util.Scanner.next(Scanner.java:1594)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2212)
    at ErrorExample.main(ErrorExample.java:13)
```

Input

55
33

Note that the test input contains 2 numbers (55 and 33).

But the program reads 3 values from the user (because it reads **num1** twice).

On line 10, the program reads **55** and stores it into **num1**.

On line 11, the program reads **33** and stores it into **num1** (overwriting 55).

So when the program reaches line 13, and tries to read a value for **num2**, both of the supplied input values have already been read, and there is no input value left to read.

Fix this error by eliminating the second read for **num1** on line 11.
so the code on line 13 will have a value to read.

```

public static void main(String[] args) {
    Scanner scnr = new Scanner(System.in);
    int num1;
    int num2;
    int sum;
    System.out.println("Enter first number");
    num1 = scnr.nextInt();
    System.out.println("Enter second number");
    num2 = scnr.nextInt();
    sum = num1 + num2;
    System.out.println("sum is " + sum);
}

```

After fixing the code, the error message will no longer appear, and the testing will proceed:

1: Compare output ^

1 / 1

Compare output

Input

55
33

Your Output

Expected output

1. Enter first number
2. Enter second number
3. sum is 88
4.

1. Enter first number
2. Enter second number
3. sum is 88
4.